



Universität Augsburg
Mathematisch-Naturwissenschaftlich-
Technische Fakultät

Applications of static and measurement based code analysis

- From computational physics to
embedded systems programming -

Jonas Kell
University of Augsburg
Chair for theoretical Physics III

28th of January 2026

2026-01-28

IMPRS Application Talk

1. Thank you for the opportunity to present my work and my plans for my future PhD research
2. I just want to give you a brief introduction: I am Jonas Kell, have studied Physics and Computer Science as a double Bachelor and Physics with a minor in Computer Science as a Master at the University of Augsburg.

What to expect from this talk

- Jonas Kell
 - University of Augsburg, Bavaria Germany
- Bachelor of Physics
- Bachelor of Computer Science
- Master of Physics
 - Minor in Computer Science
 - Focus on theoretical, computational many-body Physics



2026-01-28

IMPRS Application Talk

What to expect from this talk

1. Details about my academic career - Not the most typical Computer Scientist
2. This is not a typical paper or master-thesis presentation. While I would love to talk to you about the intricacies of fine tuning model parameters for physics-aware neural networks or the challenges of efficiently expressing analytic operator calculations in code, I realize that I can not rely on you all having heard highly specific master-level Physics lectures.
3. Even though I was told, that “The audience will be mostly from [my] area”, as this is a program for PhDs in Computer Science I will have to assume that I can not just do my presentations like I am used to from the Physics department
4. The tight timeframe of 15 minutes sadly also does not allow me to introduce the relevant facts to dive in sufficiently deep. So I have decided on a different type of presentation:
5. I will give you a high-level overview of what I been doing and focus more on the parts that are relevant or easily understood by a Computer Scientist. But most importantly this talk is going to motivate, my choice to try to obtain my PhD in Computer Science, rather than Physics

What to expect from this talk

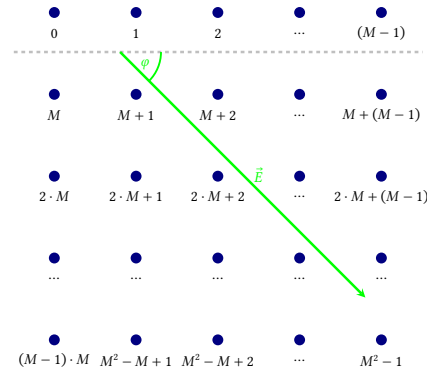
- Jonas Kell
 - University of Augsburg, Bavaria Germany
- Bachelor of Physics
- Bachelor of Computer Science
- Master of Physics
 - Minor in Computer Science
 - Focus on theoretical, computational many-body Physics

What does a theoretical Physicist do?

Introduction: Work of a computational Physicist

■ Condensed matter Physics

- Particles at discrete positions on a lattice
- Interaction governed by the *Hamiltonian*
- Particle-type has influence on the behavior



1. Without knowing the pre-requisites it will not be possible to understand most of the details. This is fine. I want to relate some High Level Concepts and open your mind to the concept, that several tools and workflows might seem “antiquated” or “solved” to an experienced Computer Scientist. At the same time this can not be said for the general Physics researcher. But it is exactly the same but in reverse for commutation-relations or magnetic interaction.
2. You never need to read formulas or the exact content of code on the following slides
3. In the case of my master-thesis: Hubbard model Hamiltonian
4. Square lattice with an external electrical field

What does a theoretical Physicist do?

Introduction: Work of a computational Physicist

$$[\hat{h}_l, \hat{h}_m] = 0$$

$$[\hat{h}_l^\dagger, \hat{h}_m^\dagger] = 0$$

$$[\hat{h}_l, \hat{h}_m^\dagger] = (1 - 2 \cdot \hat{h}_m^\dagger \hat{h}_m) \cdot \delta_{l,m}$$

$$(\hat{h}_l^\dagger \hat{h}_l)^n = \hat{h}_l^\dagger \hat{h}_l \quad \forall n \in \mathbb{N}$$

$$\hat{h}_m^{\dagger I}(t) = e^{i \cdot \varepsilon_m \cdot t} \left(1 + (e^{iU \cdot t} - 1) \hat{d}_m^{\dagger S} \hat{d}_m^S \right) \hat{h}_m^{\dagger S}$$

$$\hat{h}_m^I(t) = e^{-i \cdot \varepsilon_m \cdot t} \left(1 + (e^{-iU \cdot t} - 1) \hat{d}_m^{\dagger S} \hat{d}_m^S \right) \hat{h}_m^S$$

$$\hat{d}_m^{\dagger I}(t) = e^{i \cdot \varepsilon_m \cdot t} \left(1 + (e^{iU \cdot t} - 1) \hat{h}_m^{\dagger S} \hat{h}_m^S \right) \hat{d}_m^{\dagger S}$$

$$\hat{d}_m^I(t) = e^{-i \cdot \varepsilon_m \cdot t} \left(1 + (e^{-iU \cdot t} - 1) \hat{h}_m^{\dagger S} \hat{h}_m^S \right) \hat{d}_m^S$$



2026-01-28

IMPRS Application Talk

└ Introduction: Work of a computational Physicist

└ What does a theoretical Physicist do?

1. Particles: So called *Hard-Core Bosons*
2. All properties are reflected, if represented by operators that obey the commutation-relations listed on the left
3. Appear in *operator-strings* (seen on the right), because of form after distributing out the product of sums, you get chains of operators that need to be sorted and simplified

What does a theoretical Physicist do?
Introduction: Work of a computational Physicist

$$\begin{aligned} [\hat{h}_l, \hat{h}_m] &= 0 & \hat{h}_m^I(t) &= e^{i \cdot \varepsilon_m \cdot t} \left(1 + (e^{iU \cdot t} - 1) \hat{d}_m^{\dagger S} \hat{d}_m^S \right) \hat{h}_m^S \\ [\hat{h}_l^\dagger, \hat{h}_m^\dagger] &= 0 & \hat{h}_m^I(t) &= e^{-i \cdot \varepsilon_m \cdot t} \left(1 + (e^{-iU \cdot t} - 1) \hat{d}_m^{\dagger S} \hat{d}_m^S \right) \hat{h}_m^S \\ [\hat{h}_l, \hat{h}_m^\dagger] &= (1 - 2 \cdot \hat{h}_m^\dagger \hat{h}_m) \cdot \delta_{l,m} & \hat{d}_m^{\dagger I}(t) &= e^{i \cdot \varepsilon_m \cdot t} \left(1 + (e^{iU \cdot t} - 1) \hat{h}_m^{\dagger S} \hat{h}_m^S \right) \hat{d}_m^{\dagger S} \\ (\hat{h}_l^\dagger \hat{h}_l)^n &= \hat{h}_l^\dagger \hat{h}_l \quad \forall n \in \mathbb{N} & \hat{d}_m^I(t) &= e^{-i \cdot \varepsilon_m \cdot t} \left(1 + (e^{-iU \cdot t} - 1) \hat{h}_m^{\dagger S} \hat{h}_m^S \right) \hat{d}_m^S \end{aligned}$$

What does a theoretical Physicist do?

Introduction: Work of a computational Physicist

$$\begin{aligned}
 \hat{V}^I(t) &= \left\{ \hat{V}^S \right\}^I(t) = -J \cdot \sum_{[l,m]} \left\{ \left(\hat{h}_l^{\dagger S} \hat{h}_m^S + \hat{d}_l^{\dagger S} \hat{d}_m^S \right) \right\}^I(t) \\
 &= -J \cdot \sum_{[l,m]} \left(\hat{h}_l^{\dagger I}(t) \hat{h}_m^I(t) + \hat{d}_l^{\dagger I}(t) \hat{d}_m^I(t) \right) \\
 &= -J \cdot \sum_{[l,m]} \left[\Lambda_A(l, m, t) \cdot \hat{F}_A(l, m) + \Lambda_B(l, m, t) \cdot \hat{F}_B(l, m) + \Lambda_C(l, m, t) \cdot \hat{F}_C(l, m) \right]
 \end{aligned}$$

$$\Lambda_A(l, m, t) = e^{i(\varepsilon_l - \varepsilon_m)t} \quad \hat{F}_A(l, m) = \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (1 + 2 \cdot \hat{n}_{l,\bar{\sigma}}^S \hat{n}_{m,\bar{\sigma}}^S - \hat{n}_{l,\bar{\sigma}}^S - \hat{n}_{m,\bar{\sigma}}^S)$$

$$\Lambda_B(l, m, t) = e^{i(\varepsilon_l - \varepsilon_m + U)t} \quad \hat{F}_B(l, m) = \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (\hat{n}_{l,\bar{\sigma}}^S - \hat{n}_{l,\bar{\sigma}}^S \hat{n}_{m,\bar{\sigma}}^S)$$

$$\Lambda_C(l, m, t) = e^{i(\varepsilon_l - \varepsilon_m - U)t} \quad \hat{F}_C(l, m) = \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (\hat{n}_{m,\bar{\sigma}}^S - \hat{n}_{m,\bar{\sigma}}^S \hat{n}_{l,\bar{\sigma}}^S)$$



2026-01-28

IMPRS Application Talk

└ Introduction: Work of a computational Physicist

└ What does a theoretical Physicist do?

1. After clever re-arranging, we obtain something that resembles a *Taylor-Expansion*
 - Prefactor/Coefficient analytical
 - Occupation/State dependent multiplicator (“Feature” / “Monomial”) in several orders

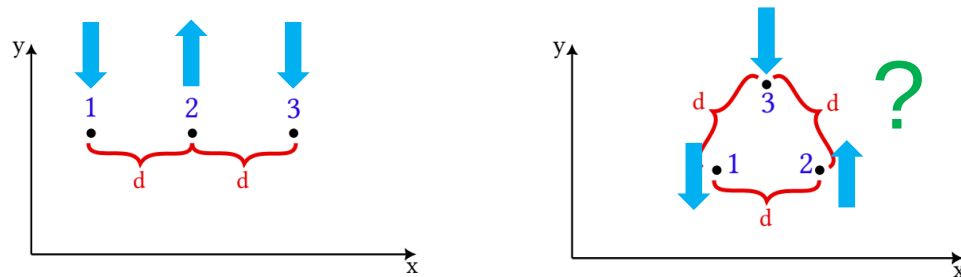
What does a theoretical Physicist do?
Introduction: Work of a computational Physicist

$$\begin{aligned}
 \hat{V}(t) &= \left\{ \hat{V}^I \right\}^I(t) = -J \cdot \sum_{[l,m]} \left\{ \left(\hat{h}_l^{\dagger S} \hat{h}_m^S + \hat{d}_l^{\dagger S} \hat{d}_m^S \right) \right\}^I(t) \\
 &= -J \cdot \sum_{[l,m]} \left(\hat{h}_l^{\dagger I}(t) \hat{h}_m^I(t) + \hat{d}_l^{\dagger I}(t) \hat{d}_m^I(t) \right) \\
 &= -J \cdot \sum_{[l,m]} \left[\Lambda_A(l, m, t) \cdot \hat{F}_A(l, m) + \Lambda_B(l, m, t) \cdot \hat{F}_B(l, m) + \Lambda_C(l, m, t) \cdot \hat{F}_C(l, m) \right]
 \end{aligned}$$

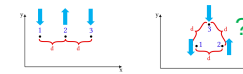
$$\begin{aligned}
 \Lambda_A(l, m, t) &= e^{i(\varepsilon_l - \varepsilon_m)t} & \hat{F}_A(l, m) &= \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (1 + 2 \cdot \hat{n}_{l,\bar{\sigma}}^S \hat{n}_{m,\bar{\sigma}}^S - \hat{n}_{l,\bar{\sigma}}^S - \hat{n}_{m,\bar{\sigma}}^S) \\
 \Lambda_B(l, m, t) &= e^{i(\varepsilon_l - \varepsilon_m + U)t} & \hat{F}_B(l, m) &= \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (\hat{n}_{l,\bar{\sigma}}^S - \hat{n}_{l,\bar{\sigma}}^S \hat{n}_{m,\bar{\sigma}}^S) \\
 \Lambda_C(l, m, t) &= e^{i(\varepsilon_l - \varepsilon_m - U)t} & \hat{F}_C(l, m) &= \sum_{\sigma \in \{\uparrow, \downarrow\}} \hat{h}_{l,\sigma}^{\dagger S} \hat{h}_{m,\sigma}^S (\hat{n}_{m,\bar{\sigma}}^S - \hat{n}_{m,\bar{\sigma}}^S \hat{n}_{l,\bar{\sigma}}^S)
 \end{aligned}$$

Where Computer Science comes into Play

Introduction: Work of a computational Physicist



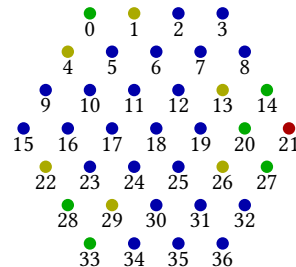
1. Why this is a computational problem can be seen be an example from my bachelor thesis
 - There an Ising-Model Hamiltonian
 - Explain Magnets orienting themselves
 - Easy in one orientation, hard in a second one - frustration



Where Computer Science comes into Play

Introduction: Work of a computational Physicist

$$\begin{array}{c}
 \langle \uparrow, \uparrow, \uparrow | \\
 \langle \uparrow, \uparrow, \downarrow | \\
 \langle \uparrow, \downarrow, \uparrow | \\
 \langle \uparrow, \downarrow, \downarrow | \\
 \langle \downarrow, \uparrow, \uparrow | \\
 \langle \downarrow, \uparrow, \downarrow | \\
 \langle \downarrow, \downarrow, \uparrow | \\
 \langle \downarrow, \downarrow, \downarrow |
 \end{array}
 \begin{pmatrix}
 |\uparrow, \uparrow, \uparrow\rangle & |\uparrow, \uparrow, \downarrow\rangle & |\uparrow, \downarrow, \uparrow\rangle & |\uparrow, \downarrow, \downarrow\rangle & |\downarrow, \uparrow, \uparrow\rangle & |\downarrow, \uparrow, \downarrow\rangle & |\downarrow, \downarrow, \uparrow\rangle & |\downarrow, \downarrow, \downarrow\rangle \\
 3 \cdot J & h & h & 0 & h & 0 & 0 & 0 \\
 h & -1 \cdot J & 0 & h & 0 & h & 0 & 0 \\
 h & 0 & -1 \cdot J & h & 0 & 0 & h & 0 \\
 0 & h & h & -1 \cdot J & 0 & 0 & 0 & h \\
 h & 0 & 0 & 0 & -1 \cdot J & h & h & 0 \\
 0 & h & 0 & 0 & h & -1 \cdot J & 0 & h \\
 0 & 0 & h & 0 & h & 0 & -1 \cdot J & h \\
 0 & 0 & 0 & h & 0 & h & h & 3 \cdot J
 \end{pmatrix}$$



1. Easy:

- Write into matrix
- Get Eigenvalues
- Profit

2. Problem: Exponential Complexity

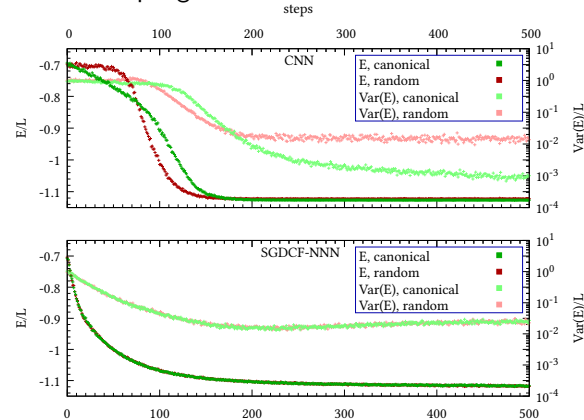
3. And: the interesting lattices get a LOT more complicated than the 3 sites from earlier

4. What is this needed for? Actually a general concept that has applications to quantum simulation, material property simulation and prediction, state of the art atom-wise manipulation and more

Modern Methods instead of brute-force Computation

Machine-Learning for Physics

- Utilization of the *universal approximator* property of neural networks
 - “Training” on some states to obtain approximate solutions for all
- Monte-Carlo sampling



1. So pop quiz: what do you do, if you have heaps of data (in our case many states) and you want to extract a ground truth in simplified and compressed representation? - Machine Learning
2. In this case in combination with Monte-Carlo sampling
3. Sadly no time to explain all the research, but in short
 - It works and has multiple applications (that was known earlier)
 - Here we see (new realization) that it can improve encoding and defect stability through usage of our graph transformers
 - Can lessen storage requirements (smaller models) through usage of symmetric graph attention
 - Here: SGDCF-NNN = Symmetric Graph Depthwise Conformer

Type Annotations and strongly typed Systems

Machine-Learning for Physics

```
446 class Metaformer(nn.Module):
447     """Metaformer architecture, based on the experiments conducted for graph-image-processing
448
449     Initialization arguments:
450     | * ``lattice_parameters``: Info on the shape of the input used for patch-embedding
451
452     """
453
454     lattice_parameters: LatticeParameters
455     embed_dim: int = 60 # ! must be divisible by num_heads
456     embed_mode: Literal[
457         "duplicate_spread", "duplicate_nn", "duplicate_nnn"
458     ] = "duplicate_spread"
459     depth: int = 12
460     mlp_ratio: float = 4.0
461     norm_layer: Callable = nn.LayerNorm
462     act_layer: Callable = nn.gelu
463     # token mixing
464     token_mixer: Literal[
465         "attention",
466         "pooling",
467         "convolution",
468     ] = "attention"
469     mixing_symmetry: Literal["arbitrary", "symm_nn", "symm_nnn"] = "arbitrary"
470     # attention specific arguments
471     num_heads: int = 6
472     qkv_bias: bool = True
473     ansatz: Literal[
474         "single-real", "single-complex", "split-complex", "two-real"
475     ] = "single-real"
476     head: Literal["act-fun", "cnn"] = "act-fun"
```

1. List of elements to use (that may seem obvious) will be listed on the following slide - but all are in general STATIC code analysis methods
2. _____
3. Untypical that so much boilerplate is written in a language that does not enforce this. But this is invaluable to have - I love languages that enforce strict typing
 - (Maybe becoming less typical with the widespread use of language models, that struggle with complex typing situations)

Type Annotations and strongly typed Systems

Machine-Learning for Physics

- Type annotations with the typing package
- Python linting settings
- (Doc) comments

```
# get model + Variational wave function
seed = 1234
if ansatz in ["single-real", "single-complex", "split-complex"]:
    model = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    psi = jVMC.vqs.NQS(model, seed=seed, batchSize=nqs_batch_size)
elif ansatz == "two-real":
    model1 = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    model2 = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    psi = jVMC.vqs.NQS((model1, model2), seed=seed, batchSize=nqs_batch_size)
else:
    raise RuntimeError(f"Ansatz '{ansatz}' ist not supported")
```



2026-01-28

IMPRS Application Talk

Machine-Learning for Physics

Type Annotations and strongly typed Systems

1. To some parts obvious for computer scientists, but actually very uncommon in productive physics research. No reproducibility, no documentation just a collection of jupyter notebooks and single-file scripts
2. Most of the time not even version control...



Test driven Development

Machine-Learning for Physics

✓  computation-scripts

-  compareenergyvariance.py
-  comparehamiltonians.py
-  compareobservables.py
-  comparevcn.py
-  comparevcnenergymeasure.py
-  concurrence.py
-  hamiltonian.py
-  measurements.py
-  observables.py
-  plot.py



2026-01-28

IMPRS Application Talk

└ Machine-Learning for Physics

└ Test driven Development

1. Next slide and rest of the talk is going to touch the topic of optimization and alternative syntax
2. Most importantly when doing so, it is imperative to assure the code does indeed do the same, just with a different timing characteristic
3. We achieve this with test fixtures (`compare....py`)

Test driven Development
Machine-Learning for Physics

└ computation-scripts

-  compareenergyvariance.py
-  comparehamiltonians.py
-  compareobservables.py
-  comparevcn.py
-  comparevcnenergymeasure.py
-  concurrence.py
-  hamiltonian.py
-  measurements.py
-  observables.py
-  plot.py

Analytical Calculations with Math-Manipulator

A personalized computer algebra system

- Built from the need to re-order non-commutative operator strings
- Open Source and web based
- Possibility to practice static code analysis and compiler components

Available here

<https://jonas-kell.github.io/math-manipulator/>



2026-01-28

IMPRS Application Talk

└ A personalized computer algebra system

└ Analytical Calculations with Math-Manipulator

1. Why is this relevant?
 - Very interesting technology and methodology required
 - Realizing I prefer building the tools for such analyses to DOING the analyses
2. Fully typed (Typescript), but definitely limited by the capabilities of Typescript over a true systems programming language

- Built from the need to re-order non-commutative operator strings
- Open Source and web based
- Possibility to practice static code analysis and compiler components

A personalized computer algebra system

The handy "Distribute" function can be used to expand products of sums.
After the terms have been fully expanded, the "Eliminate Canceling Terms" function of sums is kicked into action.
In this case "Order Operator Strings" needed to be executed twice manually (the author of the tool mainly uses it for non-commutative operators, so ordering per default is not such a clean idea).

UNIA

- └ A personalized computer algebra system

└ Analytical Calculations with Math-Manipulator

1. Example: Distribution

Analytical Calculations with Math-Manipulator

A personalized computer algebra system

`-2 c l c#m -1 c l c#n d l d#m d#l d#n + 2 c l c#n d l d#l + 1 c l c#n d#m d#m`

$$\begin{aligned} &((-2 \cdot c_l c_m^l) + (-1 \cdot c_l c_m^l d_l d_m d_l^l d_m^l) + (2 \cdot c_l c_m^l d_l d_l^l) + (1 \cdot c_l c_m^l d_m d_m^l)) \\ &((-2 \cdot c_l c_m^l) + (-1 \cdot c_l c_m^l d_l d_m d_l^l d_m^l) + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (1 \cdot c_l c_m^l d_m d_m^l)) \\ &((-2 \cdot c_l c_m^l) + (-1 \cdot c_l c_m^l d_l d_m d_l^l d_m^l) + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (1 \cdot c_l c_m^l d_l d_l^l d_m d_m^l) + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (c_l c_m^l \cdot (1 + d_l^l d_l) \cdot d_m d_m^l) + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (c_l c_m^l \cdot (1 + d_l^l d_l) \cdot (1 + d_m^l d_m)) + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (c_l c_m^l \cdot 1 \cdot 1) - (c_l c_m^l \cdot 1 \cdot d_m^l d_m) - (c_l c_m^l d_l^l d_l \cdot 1) - c_l c_m^l d_l^l d_l d_m^l d_m + (2 \cdot c_l c_m^l \cdot (1 + d_l^l d_l)) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (c_l c_m^l \cdot 1 \cdot 1) - (c_l c_m^l \cdot 1 \cdot d_m^l d_m) - (c_l c_m^l d_l^l d_l \cdot 1) - c_l c_m^l d_l^l d_l d_m^l d_m + (2 \cdot c_l c_m^l \cdot 1) + (2 \cdot c_l c_m^l d_l^l d_l) + (c_l c_m^l \cdot (1 + d_m^l d_m))) \\ &((-2 \cdot c_l c_m^l) - (c_l c_m^l \cdot 1 \cdot 1) - (c_l c_m^l \cdot 1 \cdot d_m^l d_m) - (c_l c_m^l d_l^l d_l \cdot 1) - c_l c_m^l d_l^l d_l d_m^l d_m + (2 \cdot c_l c_m^l \cdot 1) + (2 \cdot c_l c_m^l d_l^l d_l) + (c_l c_m^l \cdot 1) + c_l c_m^l d_m^l d_m) \\ &\quad (-c_l c_m^l d_l^l d_l - c_l c_m^l d_l^l d_l d_m^l d_m + (2 \cdot c_l c_m^l d_l^l d_l)) \\ &\quad (c_l c_m^l d_l^l d_l - c_l c_m^l d_l^l d_l d_m^l d_m) \end{aligned}$$

Variables:

Macros:

+		
c#	bc("c"; #0)	
c	ba("c"; #0)	
d#	bc("d"; #0)	
d	ba("d"; #0)	
displayl	lmathrm[]	
displaym	mmathrm[]	
l	dist(displayl)	
m	dist(displaym)	

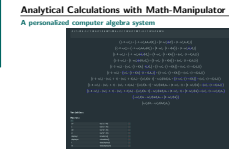


2026-01-28

IMPRS Application Talk

└ A personalized computer algebra system

└ Analytical Calculations with Math-Manipulator



1. VS-Code Extension: Permanent storage and version control

What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```
61 def part_A_flipping(  
62     Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool  
63 ) -> int:  
64  
65     if flip_up:  
66         if flip_l:  
67             Lup = not Lu  
68             Ldp = Ld  
69             Mup = Mu  
70             Mdp = Md  
71         else:  
72             Lup = Lu  
73             Ldp = Ld  
74             Mup = not Mu  
75             Mdp = Md  
76     else:  
77         if flip_l:  
78             Lup = Lu  
79             Ldp = not Ld  
80             Mup = Mu  
81             Mdp = Md  
82         else:  
83             Lup = Lu  
84             Ldp = Ld  
85             Mup = Mu  
86             Mdp = not Md  
87  
88     res = 0  
89     res += Lu and not Mu and Ld == Md  
90     res -= Lup and not Mup and Ldp == Mdp  
91     res += Ld and not Md and Lu == Mu  
92     res -= Ldp and not Mdp and Lup == Mup
```



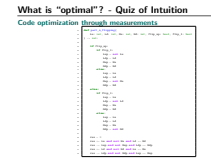
2026-01-28

IMPRS Application Talk

└ Code optimization through measurements

└ What is “optimal”? - Quiz of Intuition

1. Version 1: Naive
2. Nested ifs, much more nesting for higher orders
3. Multiple assignments and additions



What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```
5 def part_A_flipping_if_free(  
6     Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool  
7 ) -> int:  
8  
9     Lup = Lu == (not (flip_up and flip_l))  
10    Mup = Mu == (not flip_up or flip_l) # (not (flip_up and not  
11        ↳ flip_l))  
12    Ldp = Ld == (flip_up or not flip_l) # (not (not flip_up and  
13        ↳ flip_l))  
14    Mdp = Md == (flip_up or flip_l) # (not (not flip_up and not  
15        ↳ flip_l))  
16  
17    res = 0  
18    res += Lu and not Mu and Ld == Md  
19    res -= Lup and not Mup and Ldp == Mdp  
20    res += Ld and not Md and Lu == Mu  
21    res -= Ldp and not Mdp and Lup == Mup  
22  
23    return res
```



2026-01-28

IMPRS Application Talk

└ Code optimization through measurements

└ What is “optimal”? - Quiz of Intuition

1. Version 2: If-free
2. Still lots of assignments
3. More logical operations

What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```
1 def part_A_flipping_if_free(  
2     Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool  
3 ) -> int:  
4  
5     Lup = Lu == (not (flip_up and flip_l))  
6     Mup = Mu == (not flip_up or flip_l) # (not (flip_up and not  
7         ↳ flip_l))  
8     Ldp = Ld == (flip_up or not flip_l) # (not (not flip_up and  
9         ↳ flip_l))  
10    Mdp = Md == (flip_up or flip_l) # (not (not flip_up and not  
11        ↳ flip_l))  
12  
13    res = 0  
14    res += Lu and not Mu and Ld == Md  
15    res -= Lup and not Mup and Ldp == Mdp  
16    res += Ld and not Md and Lu == Mu  
17    res -= Ldp and not Mdp and Lup == Mup  
18  
19    return res
```

What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```
6 def part_A_flipping(  
7     Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool  
8 ) -> int:  
9     res = 0  
10    res += (  
11        (Lu and not Ld and not Mu and not Md)  
12        or (not Lu and Ld and not Mu and not Md)  
13        or (Lu and Ld and Mu and not Md)  
14        or (Lu and Ld and not Mu and Md)  
15    )  
16    res -= (  
17        (Lu and not Ld and not Mu and Md and not flip_up)  
18        or (not Lu and Ld and not Mu and Md and not flip_up and  
19            ~ not flip_l)  
20        or (Lu and Ld and Mu and Md and not flip_l)  
21        or (Lu and not Ld and Mu and not Md and flip_up and not  
22            ~ flip_l)  
23        or (not Lu and Ld and Mu and not Md and flip_up)  
24        or (not Lu and not Ld and not Mu and not Md and flip_l)  
25        or (Lu and not Ld and Mu and not Md and not flip_up and  
26            ~ flip_l)  
27        or (not Lu and Ld and not Mu and Md and flip_up and flip_l)  
28        or (Lu and Ld and not Mu and not Md)  
29    )  
30    return res
```

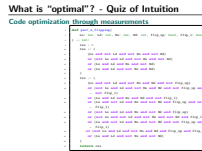


IMPRS Application Talk

└ Code optimization through measurements

└ What is “optimal”? - Quiz of Intuition

1. Version 3: assignment free, optimized
2. Pure logic
3. Logical expressions are optimized with a SAT solver to be as efficient as possible



What is “optimal”? - Answer

Code optimization through measurements

Boolean Logic optimized	23.12 s
Branch free optimized	22.02 s
Default implementation	18.8 s

- Nested If-s are indeed the fastest...
 - Might seem un-intuitive. Intuition may be wrong!
 - But probably not for every language / workload
 - Measurements required per use-case!



2026-01-28

IMPRS Application Talk

└ Code optimization through measurements

└ What is “optimal”? - Answer

1. Deep jumps are indeed fastest (no speculative execution, but seemingly advantages of binary search)
2. Pure-Logic actually only slightly slower than first optimization, even though it has a LOT more operations. So the type of operation DOES matter, even in Python

What is “optimal”? - Answer

Code optimization through measurements

Boolean Logic optimized	23.12 s
Branch free optimized	22.02 s
Default implementation	18.8 s

- Nested If-s are indeed the fastest...
 - Might seem un-intuitive. Intuition may be wrong!
 - But probably not for every language / workload
 - Measurements required per use-case!

Why pivot to Computer Science?

Lookout: Timing Analyzable Rust Compilation

- Basic lessons for a Physicist from a Computer Scientist
 - Prefer strongly typed systems
 - Perform system-specific measurements
 - Test early in and as part of the development cycle
 - Avoid boilerplate, use automatic code generation
 - Leverage static code analysis wherever possible
- I prefer developing the tools over using them
 - Start of a research proposition (initially in Augsburg)



2026-01-28

IMPRS Application Talk

└ Lookout: Timing Analyzable Rust Compilation

└ Why pivot to Computer Science?

1. Now we have learnt lessons on how code analysis combined with measurements can be of use. I realized, I enjoy working and researching this domain, so I started working on a research proposal that combines related methods with the Programming language Rust
 - Strong typing system enforced
 - Perform measurements, avoid biases from experience that may skew your view of a topic
 - Test early in and as part of the development cycle
 - Automatic code generation exists
 - Leverage static code analysis where possible

Why pivot to Computer Science?
Lookout: Timing Analyzable Rust Compilation

- Basic lessons for a Physicist from a Computer Scientist
 - Prefer strongly typed systems
 - Perform system-specific measurements
 - Test early in and as part of the development cycle
 - Avoid boilerplate, use automatic code generation
 - Leverage static code analysis wherever possible
- I prefer developing the tools over using them
 - Start of a research proposition (initially in Augsburg)

The Problem: WCET

Lookout: Timing Analyzable Rust Compilation

- WCET: *Worst Case Execution Time*
- High-volume computation mainly requires low average runtimes
 - Rare longer-running processes are not that bad
- Not so in embedded computing!
- Example: Airbag deployment system
 - Not relevant, whether 1ns or 5ns
 - BUT, we must guarantee execution faster than a certain upper bound

1. We not necessarily care about runtimes on embedded hardware
2. (Not a big difference, if the CPU busy-loops or idles or works until the end of a *tick*)
3. AS LONG, as the computation finishes faster than the available time

Advantages of Rust

Lookout: Timing Analyzable Rust Compilation

```
typedef struct
{
    int x;
} Small;

int main()
{
    Small arr[3] = {{1}, {2}, {3}};

    Small *ptr = (Small *)((char *)arr + sizeof(Small));

    printf("Second element: %d\n", ptr->x); // prints 2

    return 0;
}

#[derive(Debug)]
struct Small {
    x: i32,
}

fn main() {
    let arr = [Small { x: 1 }, Small { x: 2 }, Small { x: 3 }];

    let second = &arr[1];

    println!("Second element: {}", second.x); // prints 2
}
```



2026-01-28

IMPRS Application Talk

Lookout: Timing Analyzable Rust Compilation

Advantages of Rust

1. C:

- Slim Pointers
- Pointer casting
- Pointer Arithmetic

2. Rust:

- Smart Pointers
- Hold information about ownership, mutability, lifetime, type
- No pointer arithmetic and typecasting outside of unsafe regions

Advantages of Rust

Lookout: Timing Analyzable Rust Compilation

```
typedef struct
{
    int x;
} Small;

int main()
{
    Small arr[3] = {{1}, {2}, {3}};

    Small *ptr = (Small *)((char *)arr + sizeof(Small));

    printf("Second element: %d\n", ptr->x); // prints 2

    return 0;
}

#[derive(Debug)]
struct Small {
    x: i32,
}

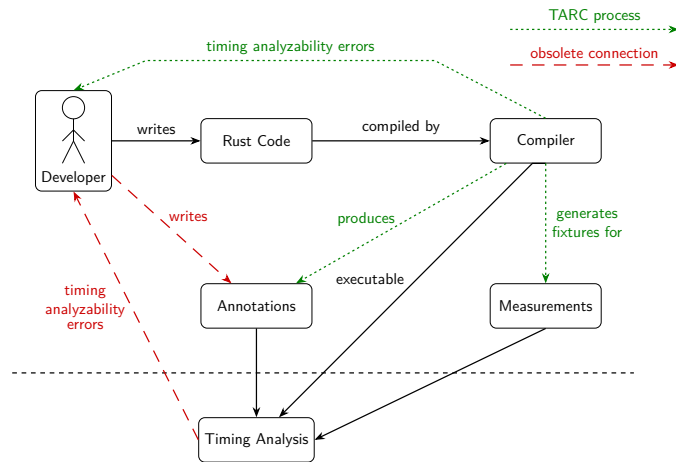
fn main() {
    let arr = [Small { x: 1 }, Small { x: 2 }, Small { x: 3 }];

    let second = &arr[1];

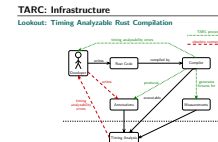
    println!("Second element: {}", second.x); // prints 2
}
```

TARC: Infrastructure

Lookout: Timing Analyzable Rust Compilation



1. Faster feedback cycle
2. Errors hard-force the developer not to use certain (possibly hard to analyze) constructs
3. Less experience with timing analysis required, but possibly also for experienced developers: biases get avoided



TARC: Minimal Example

Lookout: Timing Analyzable Rust Compilation

```
10 #[ta_analyzable()]
11 fn check_airbag_trigger_threshold() -> usize {
12     let mut sensor_values = vec![0usize; NUM_SENSORS];
13
14     for i in 0..NUM_SENSORS {
15         // first argument in ta_bounded is the code to insert,
16         // second one is a maximum runtime from an external source
17         // (measurement, analysis, calculation etc.)
18         sensor_values[i] = ta_bounded!({ read_sensor_value(i) }, 45);
19     }
20
21     let mut count = 0usize;
22     for &value in &sensor_values {
23         if value > THRESHOLD {
24             count += 1;
25         }
26     }
27
28     count
29 }
```

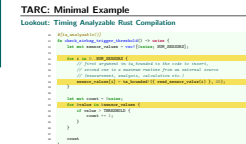


2026-01-28

IMPRS Application Talk

└ Lookout: Timing Analyzable Rust Compilation

└ TARC: Minimal Example



1. Two different kinds of macros
 - Function annotation: directly detects function scope and applies analysis to the block
 - Example in extra slides
 - Inline, part the excludes a section from analysis and gives a default “runtime” (from other source)
 - See also: a bounded loop and a vector with fixed size (possibly even more restrictive handling might be needed than allowing vec at all)
2. Possibly (not implemented as example): Zero size types

Thank you for your kind attention



UNA Universität Augsburg
Mathematisch-Naturwissenschaftlich-
Technische Fakultät



Jonas Kell
jonas-kell@web.de
www.github.com/jonas-kell

2026-01-28

IMPRS Application Talk
└ Summary & Conclusion

└ Acknowledgment

1. Thank you for your kind attention
2. Should you be interested in anything in greater detail, please feel free to reach out and also all my research is available publicly on my Github
3. Finally for the question-session please feel free to ask questions about the Timing Analyzable Rust Compilation or my past research, I would be happy to answer questions about both

Thank you for your kind attention



UNA Universität Augsburg
Mathematisch-Naturwissenschaftlich-
Technische Fakultät

Jonas Kell
jonas-kell@web.de
www.github.com/jonas-kell

References I

Summary & Conclusion

- [1] J. Kell, *Classical networks for the hubbard model with a tilted potential*, (Feb. 13, 2025) <https://github.com/jonas-kell/master-thesis-documents>.
- [2] J. Kell, *Investigation of transformer architectures for geometrical graph structures and their application to two-dimensional spin systems*, (Oct. 7, 2022) <https://github.com/jonas-kell/bachelor-thesis-documents>.
- [3] J. Kell, *Math-manipulator tool - info and repository*, (2024) <https://github.com/jonas-kell/math-manipulator>.



Extra Slides: Overview

Extra slides

- Rust Macro Test Fixtures
- Rust Macro Compile Time Errors
- Results Master Thesis
- More Math-Manipulator Features
- TARC: Timeline
- Abstract



2026-01-28

IMPRS Application Talk

└ Extra slides

└ Extra Slides: Overview

Extra Slides: Overview

Extra slides

- Rust Macro Test Fixtures
- Rust Macro Compile Time Errors
- Results Master Thesis
- More Math-Manipulator Features
- TARC: Timeline
- Abstract

Extra Slide: Rust Macro Test Fixtures

Extra slides

```
$ cargo run
Compiling rust-test v0.1.0 (/home/jonas/github/rust-test)
  Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.17s
  Running '/home/jonas/github/rust-test/target/debug/rust-test'
Entering function: check_airbag_trigger_threshold
Entering Bounded Block
Bounded Block produced Result: 1000
Exiting Bounded Block after max runtime: 45
Entering Bounded Block
Bounded Block produced Result: 1500
Exiting Bounded Block after max runtime: 45
Entering Bounded Block
....
Bounded Block produced Result: 3500
Exiting Bounded Block after max runtime: 45
Exiting function: check_airbag_trigger_threshold
```



1. Exactly function that was presented earlier
2. Print statements have been automatically inserted by the macro
3. This would be replaced with automatic measurement fixtures

Extra Slide: Rust Macro Compile Time Errors

Extra slides

```
use analyzable_macro_attribute::ta_test;

Not allowed: local assignment: rust-analyzer(macro-error)

analyzable_macro_attribute
proc_macro ta_test
View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

#[ta_test()] // this gives compilation errors
fn airbag() {
    let _ = 23 + 2;
}

// #[ta_test()] // this compiles and runs
// fn airbag() {
//     println!("test");
// }
```

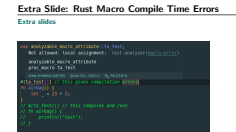


2026-01-28

IMPRS Application Talk

└ Extra slides

└ Extra Slide: Rust Macro Compile Time Errors



1.

```
use analyzable_macro_attribute::ta_test;

Not allowed: local assignment: rust-analyzer(macro-error)

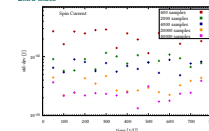
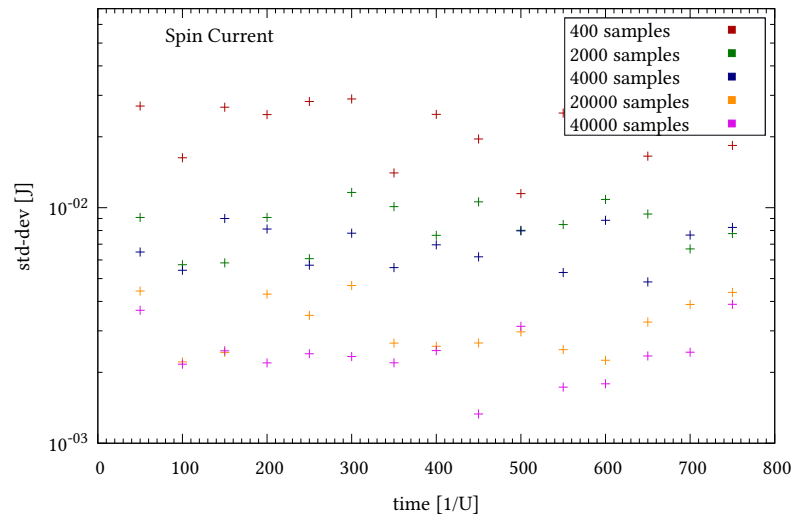
analyzable_macro_attribute
proc_macro ta_test
View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

#[ta_test()] // this gives compilation errors
fn airbag() {
    let _ = 23 + 2;
}

// #[ta_test()] // this compiles and runs
// fn airbag() {
//     println!("test");
// }
```

Extra Slide: Results Master Thesis

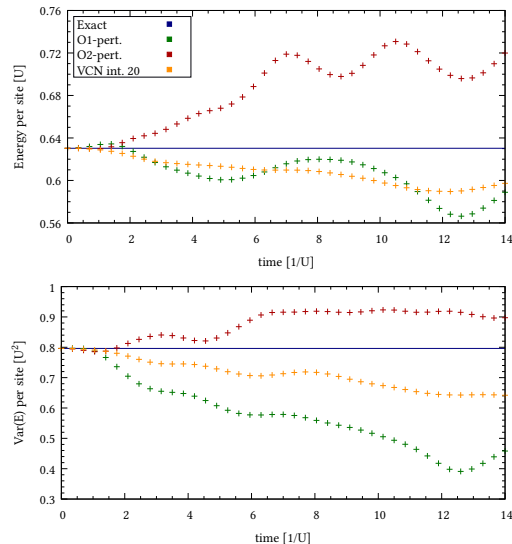
Extra slides



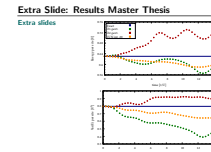
1. We see here: Error of approximation vs. exact value
2. Exact can only be obtained for small system through diagonalization
3. With more Monte-Carlo samples, we in fact converge towards the true solution in a controllable fashion

Extra Slide: Results Master Thesis

Extra slides



1. We see a plot of energy and energy variance for a large system
2. This can not be diagonalized reasonably
3. Though we know, energy must be exactly conserved
4. For that reason, the closer the values are to the one at $t=0$, the better our approximation
5. If we have this info, we know that other data (can be easily captured, once one has these values) can be trusted / error can be controlled



Extra Slide: More Math-Manipulator Features

Extra slides

- Intelligent text parsing
- Piece-wise hierarchical selection from GUI
- Function detection and custom definitions
- Constant detection
- Numerical values and variables
- Numerical constant folding
- User-definable macros
- Live variable declaration
- Bra-Ket evaluation (Physics)
- Creation- and Annihilation Operator support (Physics)
- Ordering, grouping, simplification
- Factoring and distributing
- Complex Numbers
- Rename or swap names
- Delta function evaluation
- Full LaTeX support...



2026-01-28

IMPRS Application Talk

└ Extra slides

└ Extra Slide: More Math-Manipulator Features

1. Fully Browser based
2. VS-Code extension available (support for saving files etc.)

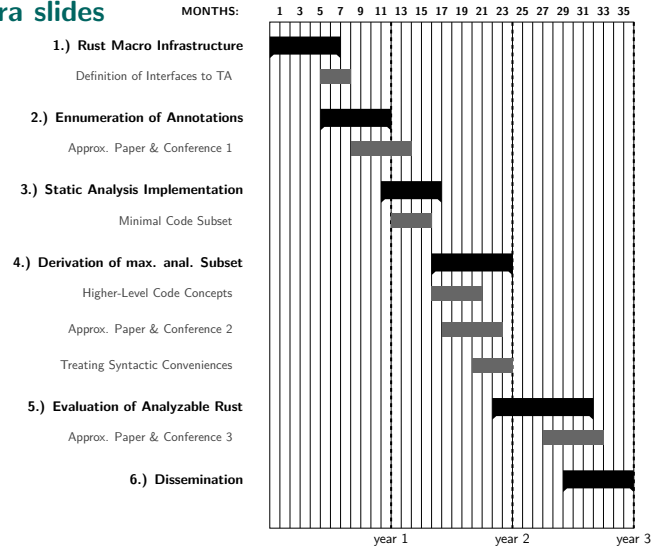
Extra Slide: More Math-Manipulator Features

Extra slides

- Intelligent text parsing
- Piece-wise hierarchical selection from GUI
- Function detection and custom definitions
- Constant detection
- Numerical values and variables
- Numerical constant folding
- User-definable macros
- Live variable declaration
- Bra-Ket evaluation (Physics)
- Creation- and Annihilation Operator support (Physics)
- Ordering, grouping, simplification
- Factoring and distributing
- Complex Numbers
- Rename or swap names
- Delta function evaluation
- Full LaTeX support...

Extra Slide: TARC: Timeline

Extra slides



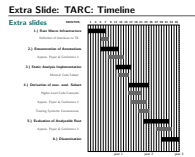
IMPRS Application Talk

Extra slides

Extra Slide: TARC: Timeline

1. Work Packages:

- Implement the macro-infrastructure within Rust
 - Robust and extensible. And solve technical challenges
- Enumerate all possible annotations
 - Depending on target platforms (cache, CPU features, etc.) other annotations and restrictions might exist
- Implement the static analysis for a minimal subset of Rust
- Derive the maximal timing analyzable subset of Rust
 - Syntactic sugar and extensible to more types than just a few basic, hardcoded ones
- Evaluate Analyzable-Rust
 - Analyze a big portion of TACLeBench



Extra Slide: Abstract

Extra slides

When learning to write code, either overall or in a previously unknown programming language, many developers might get tripped up by the colors of text and instructions in their IDE. Fresh programmers might express typical questions like “Why are the words colored all of a sudden” or “If the compiler knows that there is a semicolon missing in line 12, why do I have to place it?”. But even coding veterans might stumble upon the occasional odd “Why would this not work?” and get saved from hours of head-wrenching debugging, by the fact that the one variable is colored differently than the rest. A static code analysis tool just alerted them of the fact, that this identifier seems to be a reserved keyword in the language they were just tasked to work in.



2026-01-28

IMPRS Application Talk

└ Extra slides

└ Extra Slide: Abstract

1. When learning to write code, either overall or in a previously unknown programming language, many developers might get tripped up by the colors of text and instructions in their IDE. Fresh programmers might express typical questions like “Why are the words colored all of a sudden” or “If the compiler knows that there is a semicolon missing in line 12, why do I have to place it?”. But even coding veterans might stumble upon the occasional odd “Why would this not work?” and get saved from hours of head-wrenching debugging, by the fact that the one variable is colored differently than the rest. A static code analysis tool just alerted them of the fact, that this identifier seems to be a reserved keyword in the language they were just tasked to work in.
With a brief introduction to the programming work a computational physicist might get confronted with, we want to explore possible uses for code analysis tooling and methods to improve the development of machine learning architectures or speed up the evaluation of expensive computational tasks. We also want to pose the question: “If there is syntax checking and automatic refactoring for code, why not for doing your algebraic derivations?”. That is what a computer algebra system is for! And it basically does the same: code analysis, just on your mathematical formulas.
This train of thought motivates our research proposal called *Timing Analyzable Rust Compilation*, that suggests a toolchain targeted at improving the timing analyzability of code for embedded systems. By providing feedback about timing analyzability already during the compilation step, we plan to streamline the code for downstream tools while it is still in the high-level language. All while generating annotation symbols and meta information for external calculations of the Worst Case Execution Time (WCET) or ensuring your functions are secure against timing-based side-channel attacks.

Extra Slide: Abstract

Extra slides

When learning to write code, either overall or in a previously unknown programming language, many developers might get tripped up by the colors of text and instructions in their IDE. Fresh programmers might express typical questions like “Why are the words colored all of a sudden” or “If the compiler knows that there is a semicolon missing in line 12, why do I have to place it?”. But even coding veterans might stumble upon the occasional odd “Why would this not work?” and get saved from hours of head-wrenching debugging, by the fact that the one variable is colored differently than the rest. A static code analysis tool just alerted them of the fact, that this identifier seems to be a reserved keyword in the language they were just tasked to work in.

Extra Slide: Abstract

Extra slides

With a brief introduction to the programming work a computational physicist might get confronted with, we want to explore possible uses for code analysis tooling and methods to improve the development of machine learning architectures or speed up the evaluation of expensive computational tasks.

We also want to pose the question: “If there is syntax checking and automatic refactoring for code, why not for doing your algebraic derivations?”. That is what a computer algebra system is for! And it basically does the same: code analysis, just on your mathematical formulas.

This train of thought motivates our research proposal called *Timing Analyzable Rust Compilation*, that suggests a toolchain targeted at improving the timing analyzability of code for embedded systems. By providing feedback about timing analyzability already during the compilation step, we plan to streamline the code for downstream tools while it is still in the high-level language. All while generating annotation symbols and meta information for external calculations of the *Worst Case Execution Time* (WCET) or ensuring your functions are secure against timing-based side-channel attacks.



2026-01-28

IMPRS Application Talk

└ Extra slides

└ Extra Slide: Abstract

1. When learning to write code, either overall or in a previously unknown programming language, many developers might get tripped up by the colors of text and instructions in their IDE. Fresh programmers might express typical questions like “Why are the words colored all of a sudden” or “If the compiler knows that there is a semicolon missing in line 12, why do I have to place it?”. But even coding veterans might stumble upon the occasional odd “Why would this not work?” and get saved from hours of head-wrenching debugging, by the fact that the one variable is colored differently than the rest. A static code analysis tool just alerted them of the fact, that this identifier seems to be a reserved keyword in the language they were just tasked to work in.

With a brief introduction to the programming work a computational physicist might get confronted with, we want to explore possible uses for code analysis tooling and methods to improve the development of machine learning architectures or speed up the evaluation of expensive computational tasks.

We also want to pose the question: “If there is syntax checking and automatic refactoring for code, why not for doing your algebraic derivations?”. That is what a computer algebra system is for! And it basically does the same: code analysis, just on your mathematical formulas.

This train of thought motivates our research proposal called *Timing Analyzable Rust Compilation*, that suggests a toolchain targeted at improving the timing analyzability of code for embedded systems. By providing feedback about timing analyzability already during the compilation step, we plan to streamline the code for downstream tools while it is still in the high-level language. All while generating annotation symbols and meta information for external calculations of the *Worst Case Execution Time* (WCET) or ensuring your functions are secure against timing-based side-channel attacks.

Extra Slide: Abstract

Extra slides

With a brief introduction to the programming work a computational physicist might get confronted with, we want to explore possible uses for code analysis tooling and methods to improve the development of machine learning architectures or speed up the evaluation of expensive computational tasks.
We also want to pose the question: “If there is syntax checking and automatic refactoring for code, why not for doing your algebraic derivations?”. That is what a computer algebra system is for! And it basically does the same: code analysis, just on your mathematical formulas.
This train of thought motivates our research proposal called *Timing Analyzable Rust Compilation*, that suggests a toolchain targeted at improving the timing analyzability of code for embedded systems. By providing feedback about timing analyzability already during the compilation step, we plan to streamline the code for downstream tools while it is still in the high-level language. All while generating annotation symbols and meta information for external calculations of the *Worst Case Execution Time* (WCET) or ensuring your functions are secure against timing-based side-channel attacks.